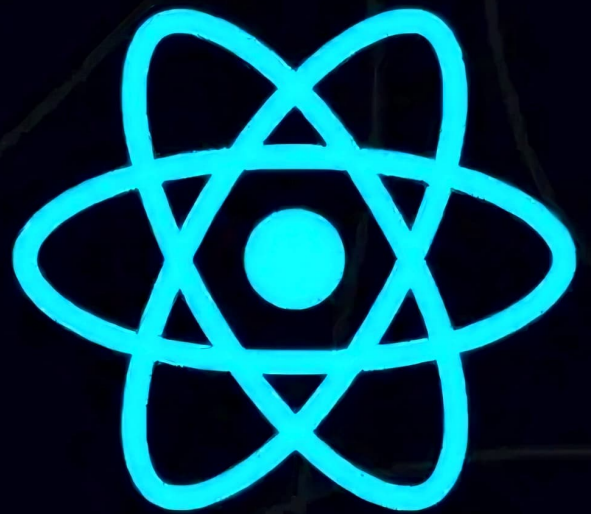
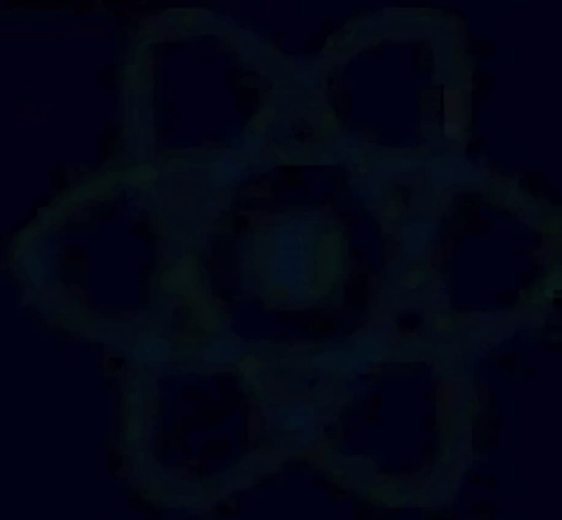


useEffect **Hook** **in React**



The **useEffect** hook in React allows you to perform **side effects** in your components. A side effect could be **data fetching**, setting up a **subscription**, or manually changing the DOM.

Think of **useEffect** as a way to tell React: "After rendering, please do this task."



Syntax



```
useEffect(() => {  
  // Code to run after the component renders  
});
```

Side Effects: Anything that affects something **outside of the component** is considered a side effect. This can include network requests, subscriptions, or logging.

Dependencies: You can control when **useEffect** runs by passing an array of **dependencies** as the second argument. The effect will only re-run if any of the dependencies change

Simple Example: Logging a Message



```
import React, { useEffect } from 'react';

function MyComponent() {
  useEffect(() => {
    console.log('Component rendered or updated');
  });

  return <div>Hello, World!</div>;
}
```

This example logs a message to the console every time the component renders.

Example with Dependencies: Data Fetching

```
import React, { useState, useEffect } from 'react';

function DataFetcher() {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch('https://api.example.com/data')
      .then(response => response.json())
      .then(data => setData(data));
  }, []); // Empty array means this effect runs only once

  return <div>Data: {data}</div>;
}
```

This example fetches data from an API and only runs the effect when the **component mounts** (i.e., the dependencies array is empty).

Example with Cleanup: Timer

```
import React, { useEffect } from 'react';

function TimerComponent() {
  useEffect(() => {
    const timer = setInterval(() => {
      console.log('Tick');
    }, 1000);

    // Cleanup function
    return () => clearInterval(timer);
  }, []); // Empty array means this effect runs only once

  return <div>Timer is running</div>;
}
```

If your effect involves setting up a resource (like a timer), you can **clean** it up to prevent **memory leaks**.

In this timer example, the **useEffect** sets up an interval that logs "Tick" every second. The returned function from **useEffect** **cleans** up the interval when the **component unmounts**.

Summary:

- **Runs after every render:** By default, `useEffect` runs after every render.
- **Dependencies Array:** The second argument to `useEffect` controls when the effect runs. If it's empty, the effect runs only once after the initial render.
- **Cleanup:** Return a function from `useEffect` to clean up resources when the component unmounts or before running the effect again.